

Análise de métodos numéricos computacionais aplicados à resolução de sistemas lineares

Walace Leal Bastos Pereira¹

¹Instituto de Matemática e Estatística

Universidade do Estado do Rio de Janeiro (UERJ) – Rio de Janeiro, RJ – Brasil

walace.pereira@trf2.jus.br

Abstract. *This work analyzes the effect of computational errors in numerical methods applied to the resolution of linear systems. We compare 3 direct methods (Gaussian Elimination, Gauss-Jordan and LU) and 3 iterative methods (Gauss-Seidel, Jacobi and SOR).*

Resumo. *Este trabalho analisa o efeito dos erros computacionais em métodos numéricos aplicados à resolução de sistemas lineares. Comparamos 3 métodos diretos (Eliminação Gaussiana, Gauss-Jordan e LU) e 3 métodos iterativos (Gauss-Seidel, Jacobi e SOR).*

1. Abordagem

1.1. Ambiente de Testes

Para este trabalho, desenvolvi um pequeno simulador web. Ele me permite visualizar o comportamento de cada algoritmo com diferentes parâmetros. Ele está disponível em <https://uerj.walaceleal.com.br>.

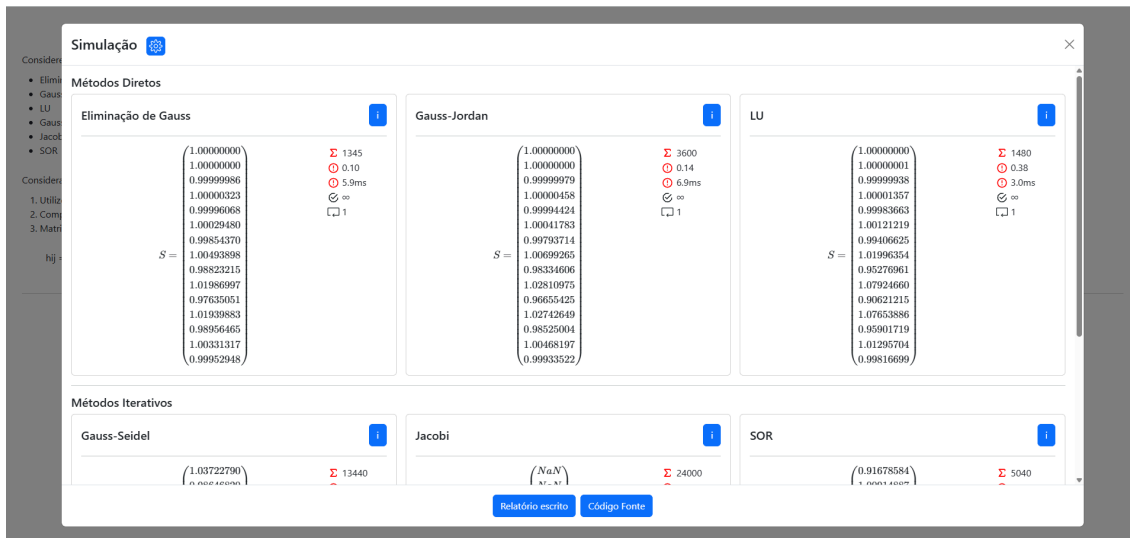


Imagem 1. Simulador

1.2. Algoritmos

Analisei o comportamento de 3 algoritmos diretos: Eliminação Gaussiana, Gauss-Jordan e LU; e de 3 algoritmos iterativos: Gauss-Seidel, Jacobi e SOR. Não implementei pivoteamento nos métodos diretos, nem analisei os aspectos teóricos ou critérios de convergência dos métodos iterativos, pois estes aspectos fogem aos conteúdos dados em sala.

1.3. Parâmetros

Nas simulações controlei a dimensão do sistema, o número máximo de iterações, a tolerância dos métodos iterativos e o parâmetro ω do algoritmo SOR e a precisão do float.

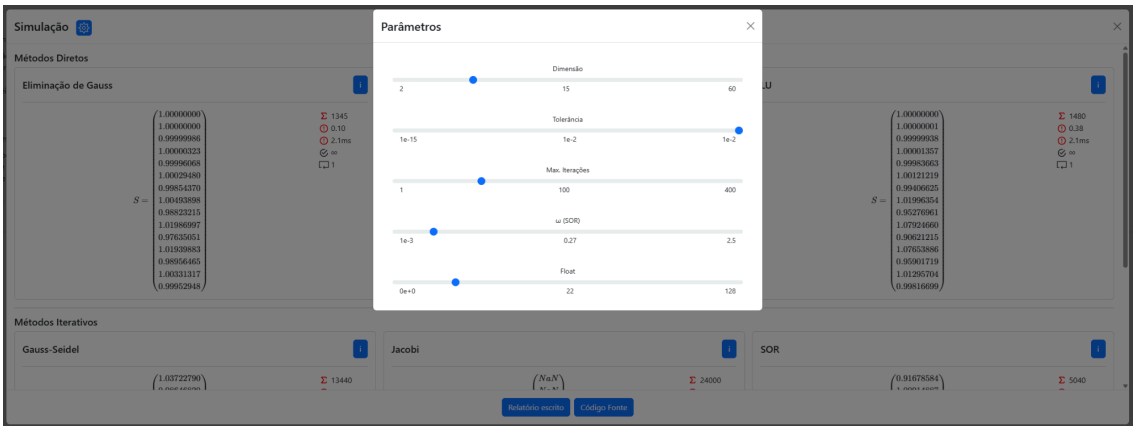


Imagem 2. Parâmetros considerados nas simulações

1.4. Métricas

Computei o número aproximado de operações, o erro cometido pelo algoritmo e o número de iterações realizadas e o tempo de execução do método.

1.5. Precisão do Float

No geral, as linguagens de programação não apresentam float com precisão dinâmica e infinita. Para realizar os testes utilizei a biblioteca javascript *Decimal*. Ela cria uma abstração e permite expandir e controlar a precisão nativa dos números.

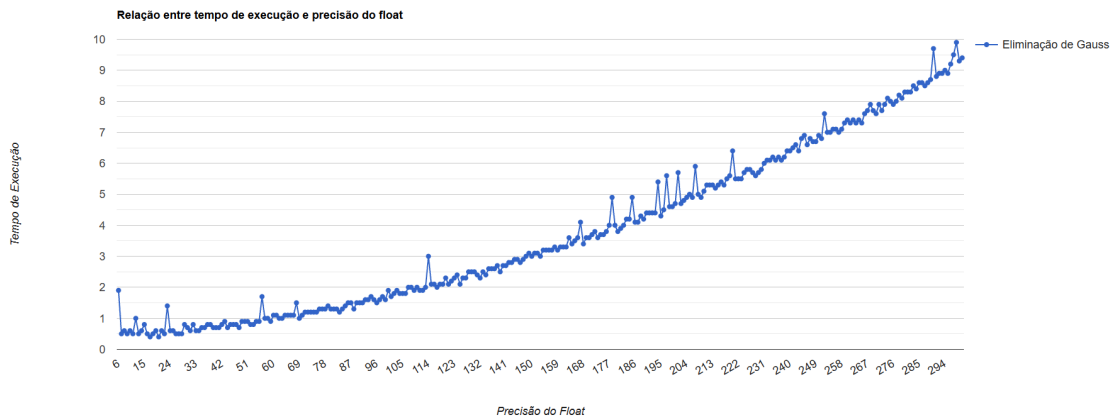


Imagem 3. Tempo de execução, alterando apenas a precisão do float

Podemos observar que, para a mesma matriz de Hilbert, aumentar a precisão tem consequências na performance.

2. Resultados

2.1. Métodos diretos

Na teoria os métodos diretos fornecem a solução deste sistema específico, contudo na prática eles são drasticamente afetados pelos erros de representação e arredondamentos inerentes dos computadores.

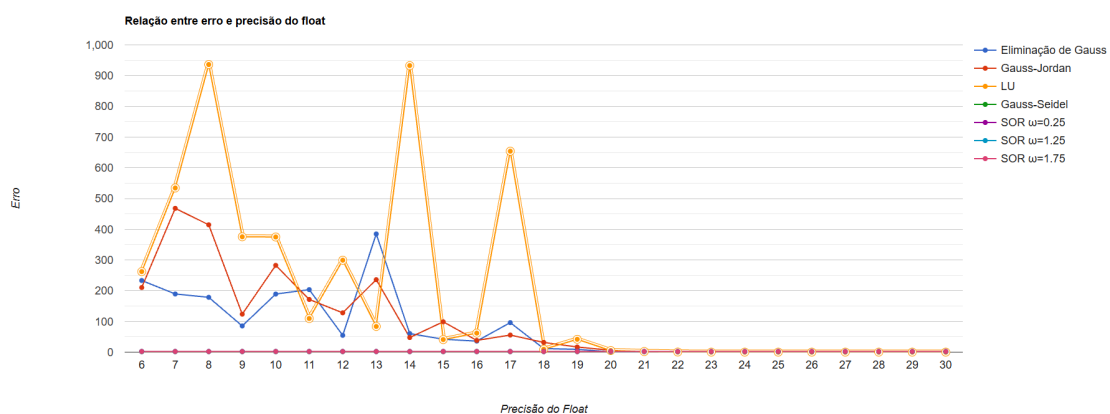


Imagem 4. Relação entre o erro e a precisão do float

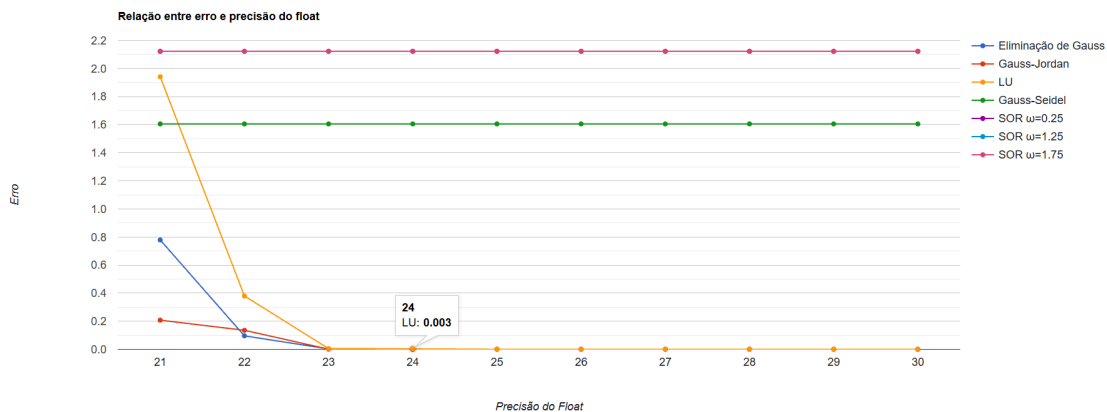


Imagem 5. Relação entre o erro e a precisão do float

Considerando a matriz de Hilbert de 15x15, os métodos diretos apresentam resultados completamente errados quando a precisão do float é baixa. Nos testes, os métodos iterativos foram menos impactados com a baixa precisão.

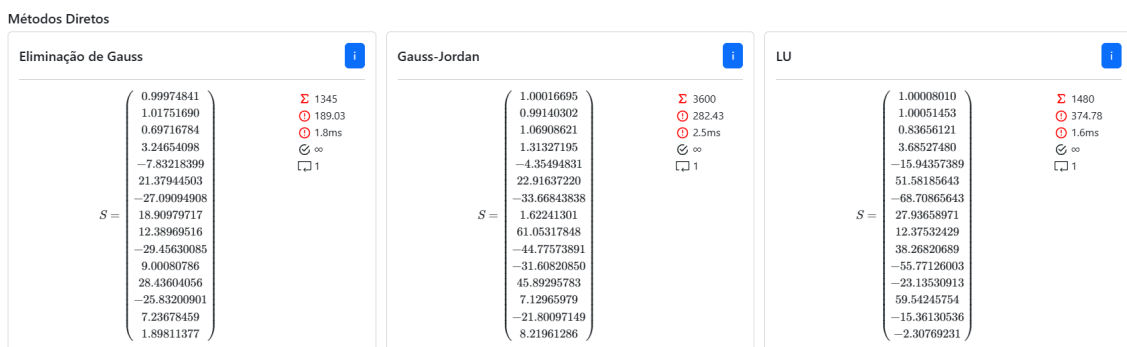


Imagem 6. Métodos diretos com precisão do *float*=8

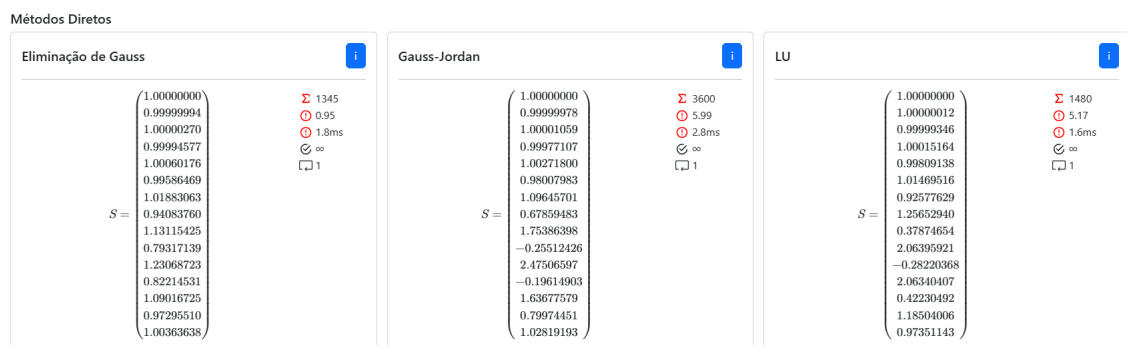


Imagem 7. Métodos diretos com precisão do *float*=20

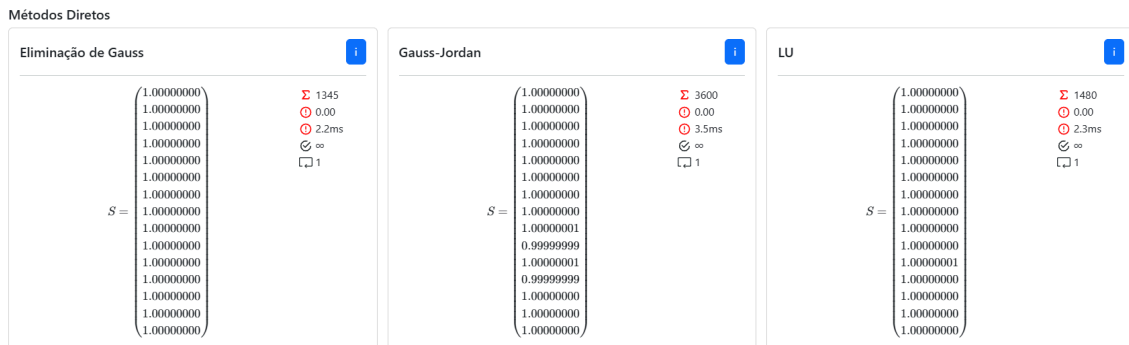


Imagem 8. Métodos diretos com precisão do *float=28*

Com base nas *imagens 5 e 6*, destacaria que a *Eliminação Gaussiana* leva ligeira vantagem em precisões intermediárias. Mas esta diferença é irrelevante quando a precisão do float é adequada, como na *imagem 7*.

2.2. Métodos iterativos

Nos testes, verifiquei que os métodos iterativos são menos afetados pelos erros inerentes dos computadores, contudo o tempo de convergência é drasticamente maior e o erro é maior quando comparado com o resultado dos métodos diretos.

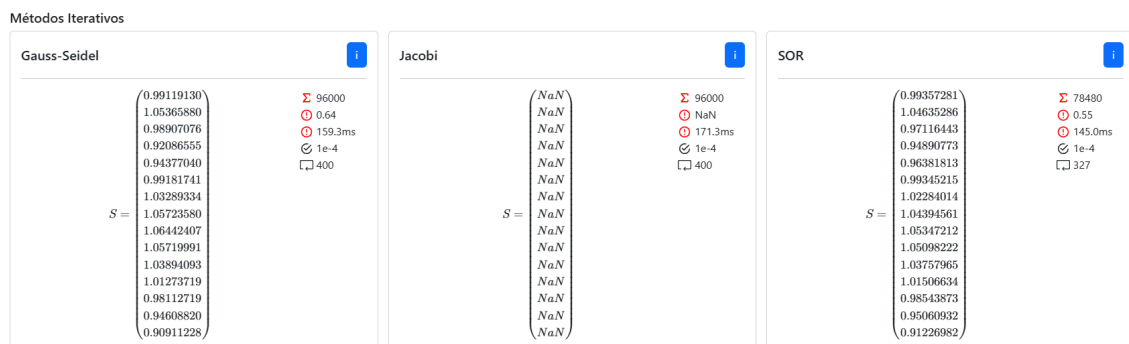


Imagem 9. Comparação entre os métodos iterativos

No caso concreto, observei que o método SOR com $\omega=0.27$ funcionou melhor que Gauss-Seidel e que Jacobi divergiu para todas matrizes de Hilbert que eu testei maiores que 2x2.

3. Conclusão

Com base nos testes, concluo que qualquer método direto com *precisão do float ajustada* resolverá muito bem o caso concreto: “Resolver o sistema com uma matriz de Hilbert **15x15**”.

Contudo vale salientar, que esta resposta só é válida para o caso concreto.

À medida que a Matriz de Hilbert cresce, a precisão numérica para utilizar métodos diretos também aumenta. Devido a limitações de memória e processamento, talvez não seja possível utilizar estes métodos mesmo com bibliotecas como o Decimal.

Neste momento, eu utilizaria o método SOR com $\omega=0.27$, pois nos testes se mostrou ligeiramente melhor que o Gauss-seidel na solução desta classe de sistemas.

Extrapolando o caso concreto, consegui resolver o sistema para uma matriz de Hilbert 120x120 com método *SOR*. Mas nem mesmo com a precisão *float=120*, obtive uma resposta válida da *Eliminação Gaussiana*.

4. Melhorias

Entendo que o simulador online é incrível para visualizar os resultados, contudo o navegador tem uma série de limitações para cálculos mais pesados. Uma melhoria que vislumbro é utilizar uma arquitetura cliente-servidor: Onde a computação das soluções é feita por uma API e apenas a exibição é feita no navegador.

